Name : Muhammad Owais
Reg No : FA24-BSE-054
Instructor : Sir Ehzaz Mustafa
Lab #5:

#Infix to Postfix using Stack :

```java
import java.io.IOException;
import java.util.Scanner;
public class IntoPost {
    private stack theStack;
    private String input;
    private String output = "";
    public IntoPost(String in){
        input = in;
        int stackSize = input.length();
        theStack = new stack(stackSize);
    }
    public String doTrans(){
        for(int j=0;j<input.length();j++){
            char ch = input.charAt(j);
            switch(ch){
                case '+':
                case '-':
                    gotOper(ch,prec1: 1);
                    break;
                case '*':
                case '/':
                    gotOper(ch,prec1: 2);
                    break;
                case '(':
                    theStack.push(ch);
                    break;
                case ')':
                    gotParen(ch);
                    break;
                default:
                    output = output + ch;
                    break;
            }
        }
        while(!theStack.isEmpty()){
            output = output + theStack.pop();
        }
        return output;
    }
```

```java
    public void gotOper(char opThis,int prec1){
        while(!theStack.isEmpty()){
            char opTop = (char) theStack.pop();
            if(opTop == '('){
                theStack.push(opTop);
                break;
            }
            else{
                int prec2;
                if(opTop == '+' || opTop == '-'){
                    prec2 = 1;
                }
                else{
                    prec2 = 2;
                }
                if(prec2 < prec1){
                    theStack.push(opTop);
                    break;
                }
                else{
                    output = output + opTop;
                }
            }
        }
        theStack.push(opThis);
    }
    public void gotParen(char ch){
        while(!theStack.isEmpty()){
            char chx = (char) theStack.pop();
            if(chx == '('){
                break;
            }
            else{
                output = output + chx;
            }
        }

    }
```

```java
    public static void main(String [] args) throws IOException {
        String input;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter infix expression: ");
        input = sc.nextLine();
        IntoPost theTrans = new IntoPost(input);
        String output = theTrans.doTrans();
        System.out.println("Postfix expression: "+output);
    }
```

```
87      class stack{
88          private int maxSize;
89          private char[] stackArray;
90          private int top;
91          public stack(int size){
92              this.maxSize = size;
93              stackArray = new char[maxSize];
94              top = -1;
95          }
96          public boolean  isEmpty(){
97              if(top==-1){
98                  return true;
99              }
100             else{
101                 return false;
102             }
103         }       public boolean isFull(){
104             if(top==maxSize-1){
105                 return true;
106             }
107             else{
108                 return false;
109             }
110         }
111         public void push(char value){
112             if(isFull()){
113                 System.out.println("Stack is full, cannot push value");
114             }
115             else{
116                 top++;
117                 stackArray[top]=value;
118                 System.out.println("Pushed value: "+value);
119             }
120         }
121         public char pop(){
122             if(isEmpty()){
123                 System.out.println("Stack is empty, cannot pop value");
124                 return (char) -1;
125             }
126             else{
127                 char value = stackArray[top];
128                 top--;
129                 System.out.println("Popped value: "+value);
130                 return value;
131             }
132         }
133         public char peek(){
134             if(isEmpty()){
135                 System.out.println("Stack is empty, cannot peek value");
136                 return (char) -1;}
137             else{
138                 System.out.println("Top element is: "+stackArray[top]);
139                 return stackArray[top];
140             }
141         }
142     }
143 }
144
```

#OUTPUT :

```
Enter infix expression: 4*5-(60/6)+5+4*7
Pushed value: *
Popped value: *
Pushed value: -
Pushed value: (
Popped value: (
Pushed value: (
Pushed value: /
Popped value: /
Popped value: (
Popped value: -
Pushed value: +
Popped value: +
Pushed value: +
Popped value: +
Pushed value: +
Pushed value: *
Popped value: *
Popped value: +
Postfix expression: 45*606/-5+47*+
```

#Infix to Prefix :

```java
import java.util.Scanner;
public class intopre {
    private stack theStack;
    private String input;
    private String output = "";
    public intopre(String in) {
        input = reverseAndSwap(in);
        int stackSize = input.length();
        theStack = new stack(stackSize);
    }
    public String reverseAndSwap(String str) {
        String reversed = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (ch == '(')
                reversed += ')';
            else if (ch == ')')
                reversed += '(';
            else
                reversed += ch;
        }
        return reversed;
    }
    public String doTrans() {
        for (int j = 0; j < input.length(); j++) {
            char ch = input.charAt(j);
            switch (ch) {
                case '+':
                case '-':
                    gotOper(ch, prec1: 1);
                    break;

                case '*':
                case '/':
                    gotOper(ch, prec1: 2);
                    break;

                case '(':
                    theStack.push(ch);
                    break;

                case ')':
                    gotParen();
                    break;
                default:
                    output = output + ch;
                    break;
            }
        }
        while (!theStack.isEmpty()) {
            output = output + theStack.pop();
        }
        return reverse(output);
    }
    public void gotOper(char opThis, int prec1) {
        while (!theStack.isEmpty()) {
            char opTop = theStack.pop();
            if (opTop == '(') {
                theStack.push(opTop);
                break;
            }
            else {
                int prec2;
                if (opTop == '+' || opTop == '-')
                    prec2 = 1;
                else
                    prec2 = 2;
                if (prec2 < prec1) {
                    theStack.push(opTop);
                    break;
                }
                else {
                    output = output + opTop;
                }
            }
        }
        theStack.push(opThis);
    }
```

```
80      public void gotParen() {
81          while (!theStack.isEmpty()) {
82              char chx = theStack.pop();
83              if (chx == '(')
84                  break;
85              else
86                  output = output + chx;
87          }
88      }
89      public String reverse(String str) {
90          String rev = "";
91          for (int i = str.length() - 1; i >= 0; i--) {
92              rev += str.charAt(i);
93          }
94          return rev;
95      }
96
            Run | Debug | Run main | Debug main
97      public static void main(String[] args) {
98          Scanner sc = new Scanner(System.in);
99          System.out.print("Enter infix expression: ");
100         String input = sc.nextLine();
101         intopre theTrans = new intopre(input);
102         String output = theTrans.doTrans();
103         System.out.println("Prefix expression: " + output);
104     }
105     class stack {
106         private int maxSize;
107         private char[] stackArray;
108         private int top;
109         public stack(int size) {
110             this.maxSize = size;
111             stackArray = new char[maxSize];
112             top = -1;
113         }
114         public boolean isEmpty() {
115             return top == -1;
116         }
117         public boolean isFull() {
118             return top == maxSize - 1;
119         }
120         public void push(char value) {
121             if (!isFull()) {
122                 stackArray[++top] = value;
123             }
124         }
125         public char pop() {
126             if (!isEmpty()) {
127                 return stackArray[top--];
128             }
129             return (char) -1;
130         }
131         public char peek() {
132             if (!isEmpty()) {
133                 return stackArray[top];
134             }
135             return (char) -1;
136         }
137     }
138 }
```

#OUTPUT :